Paper 4 - Boundary Repair

CQE-paper-04
CQECMPLX Formal Paper Series
Review-facing PDF built 2026-06-12

Construction Basis

Define boundary repair as the transport operation that converts failed joins into typed constraints for the next legal route.

This PDF is the clean paper artifact. Source extracts, kernels, receipts, tool notes, workbook material, and package bindings are used as construction evidence; they are not treated as separate review-facing papers.

Evidence Inputs

- top-level review source: Papers\Source\CQE-paper-04.md
- paper kernel manifest: production\paper-kernels\papers\CQE-paper-04\paper_kernel_manifest.json
- body: production\papers\CQE-paper-04\PAPER-BODY.md
- source: production\papers\CQE-paper-04\SOURCE.md
- formal: production\papers\CQE-paper-04\01-CQE-formal\FORMAL.md
- tool: production\papers\CQE-paper-04\02-CQE-tool\TOOL.md
- tool_runner: production\papers\CQE-paper-04\02-CQE-tool\run.py
- workbook: production\papers\CQE-paper-04\03-CQE-workbook\WORKBOOK.md
- recovered_output: production\lib-forge\recovered\papers_output\CQE-paper-04.md

Paper 4 - Boundary Repair

Abstract

Paper 4 proves the boundary repair operation for the first correction chain. Paper 2 produces nonzero correction residues. Paper 3 registers those residues as axis/sheet coordinates. Paper 4 converts the registered failure into a typed, idempotent constraint that the next legal route can consume.

The theorem is deliberately precise:

```
failed join -> typed boundary repair constraint
```

It does not claim:

```
failed join -> proof
```

A repair exists only when the failure is recorded with enough information to be replayed: state, coordinate, reason, status, next legal routes, source paper, and target paper.

Claims

Claim 4.1. The two Paper 2 correction states can be converted into typed boundary repair constraints while preserving their Paper 3 coordinates.

Claim 4.2. A repaired row is a constraint, not a proof.

Claim 4.3. The repair operation is idempotent on already repaired rows.

Claim 4.4. An untyped failure such as `{"status":"failed"}` is not a repair and must be rejected.

Definitions

An **LCR state** is:

```
s = (L,C,R) in {0,1}^3
```

A **correction residue** is a Paper 2 row where:

```
corr(L,C,R) = C and not R = 1
```

A **registered coordinate** is the Paper 3 axis/sheet coordinate:

```
coord(s) = (axis(s), sheet(s))
```

A **failed join** is a correction residue that cannot be accepted by the current route as a closed proof.

A **boundary repair constraint** is a row with at least:

```
state
axis_sheet
reason
status
next_legal_routes
source_paper
target_paper
```

The required status is `constraint`.

Theorem 4.1 - Typed Boundary Repair

A failed join is repairable in the CQECMPLX paper kernel if and only if it can be converted into a typed constraint that preserves the original state, the Paper 3 coordinate, the reason for failure, and at least one next legal route.

Proof

If a failure is not recorded with state, coordinate, reason, status, and a next legal route, then the next transport has no reproducible object to consume. The failure may be observed, but it is not repaired.

If those fields are present, the next transport receives a determinate constraint. It can accept, defer, or reject the row with a receipt. The failure has therefore been repaired at the boundary level: it has become legal input to the next route without being falsely promoted to a theorem.

Idempotence follows from the definition. Applying the repair operation to an already repaired row returns the same row because the state, coordinate, reason, status, and routes are already fixed.

Concrete Repair Rows

Paper 2 supplies the correction states:

```
(0,1,0)
(1,1,0)
```

Paper 3 supplies their coordinates:

```
(0,1,0) -> (2,0)
(1,1,0) -> (3,1)
```

Paper 4 converts them to constraint rows:

```
state: original LCR state
axis_sheet: Paper 3 coordinate
reason: Paper 2 correction fired: C and not R
status: constraint
next_legal_routes: Paper 5 path-carrier intake, Paper 3 stronger theorem intake
source_paper: CQE-paper-04
target_paper: CQE-paper-05
```

Receipt

The primary executable receipt is:

```
production/formal-papers/CQE-paper-04/verify_boundary_repair.py
production/formal-papers/CQE-paper-04/boundary_repair_receipt.json
```

The receipt status is `pass`. It verifies:

```
consumes_two_paper02_correction_states = true
preserves_paper03_coordinates          = true
all_required_fields_present            = true
constraints_not_proofs                 = true
next_legal_routes_nonempty             = true
repair_is_idempotent                   = true
untyped_failure_rejected               = true
```

Falsifier

The minimal falsifier is:

```
{"status":"failed"}
```

It is rejected because it lacks the state, coordinate, reason, source, target, and next legal route required for repair.

Role in the Suite

Paper 4 is the transition from registered residue to legal transport input:

```
Paper 2 correction residue
+ Paper 3 coordinate
-> Paper 4 boundary constraint
-> Paper 5 path-carrier payload
```

The point is not to hide failure. The point is to make failure carryable.

Open Obligations

- Expose `verify_boundary_repair` through the installable kernel/API interface.
- Bind the repair-row schema to a shared obligation ledger for later papers.
- Connect boundary constraints directly to Paper 5 path-carrier payloads.
- Keep deeper oloid midpoint or Dust-pair interpretations in later papers unless their own verifiers are attached.

Conclusion

Boundary repair is the suite's first formal failure-preservation operator. A failed join becomes a typed, idempotent, replayable constraint. It remains honest because it is not proof; it is the structured object that makes the next legal proof attempt possible.